

Capstone Project II

Team No: 8

AI-Based Cyber Threat Detection & Altering System

Software Design Document

Submitted by

S NO	NAME	ENROLMENT NO
1	Anjali Nair	BT23GCS097
2	Ayushi Nawal	BT23GCS520
3	Shraddha Mandal	BT23GCS100
4	Vinisha Shah	BT23GCS202

Faculty Mentor: Mr. Manish Hurkat

DATE: (20/09/2026)

Table of Contents

1. Introduction —
   1.1 Purpose ·
   1.2 Scope ·
   1.3 Definitions, Acronyms and Abbreviations
2. System Overview —
   2.1 System Context ·
   2.2 Overall Architectural Approach ·
   2.3 Subsystem Interaction Overview ·
   2.4 Deployment Overview
3. System Architecture —
   3.1 Architectural Style ·
   3.2 Layered Architecture Description ·
   3.3 Microservice & Communication Design ·
   3.4 Component Interaction Model ·
   3.5 Deployment Mapping to Infrastructure ·
   3.6 Architectural Design Decisions
4. System Modules —
   4.1 Authentication & User Management ·
   4.2 Traffic Capture & Data Ingestion ·
   4.3 AI Detection & Scoring ·
   4.4 Alert Screening & Response Decision Support ·
   4.5 Notification Management ·
   4.6 Dashboard & Reporting ·
   4.7 Logging & Monitoring
5. Data Design —
   5.1 Database Architecture Overview ·
   5.2 Schema Design ·
   5.3–5.9 Collections ·
   5.10 Data Relationships ·
   5.11 Indexing Strategy ·
   5.12 Retention & Archival
6. Interface Design —
   6.1 User Interface Design ·
   6.2 API Design ·
   6.3 External Interfaces
7. Detailed Algorithm Design —
   7.1 Flow Feature Extraction ·
   7.2 Supervised Attack Classification ·
   7.3 Anomaly Detection & Verdict Fusion ·

7.4 Explainable Scoring ·
7.5 Alert Triage & Ranking
8. Non-Functional Design —
8.2 Performance ·
8.3 Scalability ·
8.4 Availability & Reliability ·
8.5 Maintainability ·
8.6 Usability ·
8.7 Portability
9. Security Design —
9.2 Authentication ·
9.3 Authorization/RBAC ·
9.4 Communication Security ·
9.5 Data Security ·
9.6 API Security Controls ·
9.7 Logging & Audit Security
10. Deployment & Operations —
10.1 Deployment Architecture ·
10.2 Deployable Units ·
10.3 Docker Configuration ·
10.4 CI/CD Pipeline ·
10.5 Monitoring ·
10.6 Backup & Disaster Recovery
11. Testing —
11.2 Unit ·
11.3 Integration ·
11.4 API & Service ·
11.5 ML/Detection Testing ·
11.6 Performance ·
11.7 Security ·
11.8 Reliability & Failure Recovery ·
11.9 UAT
12. References
13. Appendices —
13.1 ER Diagram ·
13.2 System Sequence Diagram ·
13.3 Detection & Alert Flow Diagram ·
13.4 Alert Status Transition Diagram ·
13.5 API Request/Response Examples

1. Introduction

PacketGuard AI is an AI-based real-time network intrusion detection and automated alerting platform designed as a web-based security operations tool for security analysts and administrators. The system is intended to reduce the manual effort involved in network traffic triage and improve the speed, accuracy and explainability of threat detection.

The system provides live traffic monitoring, hybrid ML-based detection (a supervised classifier plus an unsupervised anomaly model), explainable severity scoring, analyst-driven alert response, simulated IP blocking, threat-intelligence enrichment, notifications, dashboards, reporting, and audit logging. The design is based on the requirement that AI assists detection but never makes the final block/response decision — that decision remains under human analyst control.

1.1 Purpose

The purpose of this Software Design Document (SDD) is to describe the technical design and architecture of PacketGuard AI based on the requirements defined in the Software Requirements Specification (SRS).

The SDD translates the functional and non-functional requirements of the SRS into a structured software design that can be used by developers, ML engineers, testers, reviewers, and future maintainers. It provides the design basis for implementing:

- User registration, authentication, authorization and role-based access control.
- Network traffic capture and flow-level feature extraction.
- Streaming ingestion of flow events through a big-data message platform.
- Hybrid ML detection: supervised attack classification and unsupervised anomaly detection.
- Verdict fusion and severity scoring.
- Explainable detection through SHAP-based top contributing features.
- Threat-intelligence enrichment for suspicious IPs.
- Analyst-driven alert lifecycle management and simulated IP blocking.
- Real-time notifications.
- Analyst and administrator dashboards.
- Detection analytics and reporting.
- Audit logging and system monitoring.
- Secure deployment using containerization and Kubernetes.

The SRS establishes what the system must do; this SDD covers detailed design. It serves as the technical reference for converting requirements into implementable software components, interfaces, data structures, processing flows, security mechanisms and deployment structures.

The intended users of this SDD are:

- **Project Developers** — for implementation of frontend, backend, database and supporting services.
- **ML Engineers** — for feature extraction, detection models, explainability and verdict fusion.
- **Testers** — for deriving unit, integration, API, security, performance and ML evaluation tests.
- **Project Reviewers and Faculty** — for evaluating whether the design satisfies the approved SRS.
- **Future Maintainers** — for understanding the system structure and maintaining or extending it.

This SDD supports requirement traceability: each SRS requirement is associated with an SDD design element and a corresponding test case.

1.2 Scope

The scope of the PacketGuard AI software design covers the complete workflow from traffic capture through ML-based detection, analyst review, response actions, notifications, analytics, administration and auditing.

PacketGuard AI has two primary user roles:

1. **Security Analyst**
2. **Administrator**

The system is a self-contained web application that communicates with a small set of external services: a streaming platform (Redpanda), a threat-intelligence provider, and an email/Slack notification service.

Security Analyst Scope — live alert feed, alert detail with raw flow features and SHAP explanation, acknowledge/resolve/false-positive actions, block approval, threat-intel lookup, notifications, personal settings.

Administrator Scope — analyst account management, block-rule management, detection threshold configuration, audit-log viewing, system health monitoring, reporting.

ML Processing Scope — the ML subsystem is responsible for scoring traffic, not for independently blocking it. Scope includes: flow feature extraction from captured packets; supervised classification of attack family; unsupervised anomaly scoring against benign-only baseline traffic; verdict fusion; SHAP-based explanation generation; model versioning; and threat-intelligence enrichment of flagged IPs.

1.3 Definitions, Acronyms and Abbreviations

See SRS Section 1.3 for the full glossary (AI, ML, IDS/IPS, API, REST, JWT, RBAC, MERN, Flow, Feature extraction, Supervised/Anomaly model, Verdict fusion, SHAP, Severity score, Alert, Block rule, Threat intelligence enrichment, Big-data streaming layer, Audit log, CI/CD, SRS/SDD). Design-specific additions:

- **Layered architecture:** organizing the system into presentation, application, ML processing, data and infrastructure layers with defined responsibilities per layer.
- **Service isolation:** running each major capability (capture, ML inference, backend API, frontend) as an independently deployable container.
- **Verdict fusion rule:** the logic combining supervised and anomaly outputs into one benign/malicious decision (see SRS FR-DETECT-03).
- **Change stream:** a MongoDB feature that lets the backend react to new/updated documents in real time, used to drive Socket.io alert broadcasts.

2. System Overview

2.1 System Context

PacketGuard AI sits between raw network traffic and the human analyst. It ingests flow-level data, scores it, and presents actionable, explained alerts. It does not sit inline on the network path — it observes and recommends, and the analyst (or a future, separately-scoped enforcement layer) acts.

2.2 Overall Architectural Approach

The system is organized into six conceptual layers:

2.2.1 Data Acquisition Layer

Captures packets (live interface or `.pcap` replay), extracts flow-level features, and publishes them as JSON events to the streaming platform. Owned by the **sniffer** service.

2.2.2 Processing & AI Layer

Consumes flow events from the stream, runs the supervised classifier and anomaly model, fuses the verdict, computes SHAP explanations, and writes alerts to the database. Owned by the **ml-service**.

2.2.3 Decision & Alert Layer

Manages the alert lifecycle (new → acknowledged → resolved/blocked/false-positive), triggers notifications for critical severity, and records analyst decisions. Owned by the **backend API**.

2.2.4 Application Layer

The React dashboard: live alert feed, explanation panel, block-approval workflow, analytics views, and administration screens.

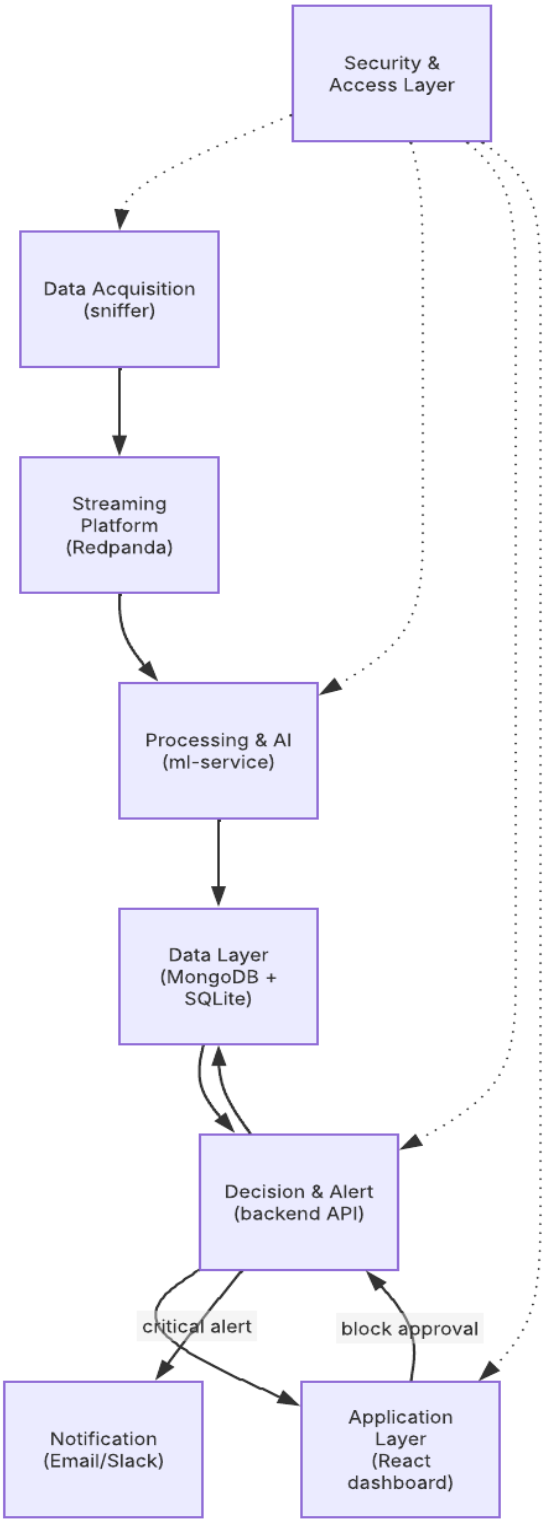
2.2.5 Data Layer

MongoDB (flows, alerts, threat-intel cache — high-volume, semi-structured) and SQLite via Prisma (users, block rules — small, relational, transactional).

2.2.6 Security & Access Layer

JWT-based authentication, role-based access control, TLS in transit, input validation, and audit logging cutting across every other layer.

2.3 Subsystem Interaction Overview



Each subsystem is a separately deployable container communicating over well-defined interfaces (Kafka-protocol streaming, REST/JSON, WebSocket), so any layer can be scaled, replaced or tested independently.

2.4 Deployment Overview

All services run as Docker containers, orchestrated locally with Docker Compose and, for the final release, with Kubernetes. A Jenkins pipeline builds, tests and deploys every service on each push. This is detailed fully in Section 10.

3. System Architecture

3.1 Architectural Style

3.1.1 Layered Architecture

The system separates presentation, application/backend, ML processing, data and infrastructure concerns so each can evolve and be tested independently.

3.1.2 Service Isolation

Each major capability — sniffer, ml-service, backend, frontend — runs as its own container with its own dependencies, so a crash or slowdown in one (e.g. the ML service) does not take down the others.

3.1.3 RESTful Communication

The frontend and backend communicate over REST/JSON for request-response operations and WebSocket (Socket.io) for real-time push. Internal streaming (sniffer → ml-service) uses the Kafka protocol over Redpanda rather than REST, since it is a continuous event flow, not a request-response interaction.

3.1.4 Containerized Deployment

Every service ships as a Docker image, composed locally with Docker Compose and deployed to Kubernetes for the final release, so the same artifacts run in development and production.

3.2 Layered Architecture Description

3.2.1 Presentation Layer

React 18 + Vite single-page application. Renders role-specific views (Analyst, Administrator), consumes the REST API for data and Socket.io for live updates.

3.2.2 Application Layer (Backend)

Node.js 24 + Express. Owns authentication, alert lifecycle, block-rule management, notification dispatch, and Socket.io broadcast. Reads/writes MongoDB (via Mongoose) and SQLite (via Prisma).

3.2.3 ML Processing Layer

Python + FastAPI. Consumes flow events from Redpanda, runs the supervised and anomaly models, computes SHAP explanations, and writes alerts directly to MongoDB (the fastest path for high-volume writes, decoupled from the backend's request/response cycle).

3.2.4 Data Layer

MongoDB for flows/alerts/threat-intel (document-oriented, high write volume, variable shape); SQLite via Prisma for users/block-rules (relational, transactional, low volume).

3.2.5 Infrastructure Layer

Docker for containerization, Redpanda for streaming, Kubernetes for orchestration, Jenkins for CI/CD.

3.3 Microservice & Communication Design

3.3.1 Frontend to Backend Communication

HTTPS REST for CRUD operations (login, alert queries, status updates); a single persistent WebSocket connection, authenticated with the same JWT, for `new-alert` and `alert-updated` push events.

3.3.2 Backend to ML Service Communication

Indirect, via MongoDB: the ML service writes alert documents; the backend watches for changes via a MongoDB change stream and re-broadcasts over Socket.io. This decouples the two services — the ML service never needs to know the backend exists.

3.3.3 Backend to Database Communication

Mongoose ODM for MongoDB (flows, alerts, threat-intel); Prisma ORM for SQLite (users, block rules). Each library is used only for its own database — no cross-database joins are attempted at the query level; the application layer joins where needed (e.g. resolving `assignedToUserId` from Mongo against the Prisma `User` table).

3.3.4 Flow Data Ingestion

The sniffer publishes flow-feature JSON events to the `network-flows` Redpanda topic. The ML service is the sole consumer group for this topic in the current design, keeping the ingestion path simple for a 3-week build.

3.3.5 Internal Security Boundaries

The streaming platform and MongoDB are not exposed outside the Docker network / Kubernetes cluster; only the backend and frontend expose ports externally, and only the backend requires authenticated requests.

3.4 Component Interaction Model

3.4.1 User Authentication Flow

Login request → backend validates credentials against Prisma `User` → issues JWT access + refresh tokens → frontend stores the access token in memory and attaches it to REST calls and the Socket.io handshake.

3.4.2 Flow Capture and Detection Processing Flow

Sniffer captures/replays traffic → extracts features → publishes to Redpanda → ML service consumes → scores (supervised + anomaly) → fuses verdict → computes SHAP → writes alert to MongoDB.

3.4.3 Notification Generation Flow

Backend's MongoDB change-stream watcher detects a new alert → emits `new-alert` over Socket.io → if severity is critical, also calls the email/Slack webhook.

3.4.4 Reporting and Analytics Flow

Frontend requests `/api/stats/summary` → backend aggregates counts from MongoDB (by severity/status) → returns a summary payload rendered as dashboard charts.

3.4.5 Logging and Audit Flow

Every ML verdict write and every analyst decision (acknowledge/resolve/block) triggers an `AuditLog` write (SQLite via Prisma) capturing actor, action, target and timestamp, independent of the primary data write, so an audit failure never blocks the primary operation but is itself logged.

3.5 Deployment Mapping to Infrastructure

3.5.1 Containerized Service Deployment

Six containers: `sniffer`, `ml-service`, `backend`, `frontend`, `mongo`, `redpanda` (SQLite lives as a file volume inside the `backend` container).

3.5.2 Network Architecture

A single Docker Compose network (or Kubernetes namespace) with internal DNS service names (`mongo`, `redpanda`, `backend`); only `frontend` (port 80/3000) and `backend` (port 5000, for direct API testing) are exposed to the host/ingress.

3.5.3 CI/CD Integration

Jenkins builds and tests each service on every push, builds and pushes Docker images, then applies Kubernetes manifests to roll out the new versions (see Section 10.4).

3.5.4 Scalability Considerations

The `backend` and `ml-service` are stateless and can run multiple replicas behind their Kubernetes Service; `ml-service` additionally uses a Horizontal Pod Autoscaler since inference is the most CPU-intensive step.

3.6 Architectural Design Decisions

3.6.1 ML Service Separation

Separating ML inference into its own Python service (rather than embedding a model in the Node.js backend) lets the ML stack use its natural ecosystem (scikit-learn, XGBoost, SHAP) and scale independently of API traffic.

3.6.2 MERN Stack Selection

MongoDB, Express, React and Node.js were chosen because they are explicitly required by the capstone brief and because MongoDB's document model fits variable-shape flow/alert data well.

3.6.3 Containerized Deployment

Docker/Kubernetes were chosen to satisfy the brief's DevOps requirements and to guarantee the same behavior in development, CI and the final demo environment.

3.6.4 Stateless API Design

The backend keeps no in-memory session state; all state lives in MongoDB/SQLite, so any backend replica can serve any request — a prerequisite for the horizontal scaling described in 3.5.4.

4. System Modules

4.1 Authentication & User Management Module

4.1.1 Purpose and Responsibilities

Authenticates analysts and administrators, issues and validates tokens, and enforces role-based access to every other module.

4.1.2 Internal Components

`authController` (register, login, refresh, logout), `authMiddleware` (JWT verification, role checks), `socketAuth` (WebSocket handshake verification).

4.1.3 API Endpoints

`POST /api/auth/register`, `POST /api/auth/login`, `POST /api/auth/refresh`, `POST /api/auth/logout`.

4.1.4 Authentication Flow

Credentials → bcrypt hash comparison against the Prisma `User` table → on success, sign a 15-minute JWT access token and a 7-day refresh token → client stores the access token in memory and the refresh token in an httpOnly cookie.

4.1.5 JWT Structure and RBAC

Payload: `{ userId, role, iat, exp }`. `authMiddleware` decodes the token, attaches `req.user`, and route-level guards check `req.user.role` against the allowed roles for that route (`analyst`, `admin`).

4.1.6 User Data Model

See Section 5.3 (`User` table, Prisma/SQLite).

4.1.7 Security Considerations

Password hashing with bcrypt (cost factor 12); account lockout after 5 failed attempts within 15 minutes; refresh tokens invalidated on logout; no role information is ever trusted from the client — it is always re-read from the verified JWT server-side.

4.2 Traffic Capture & Data Ingestion Module

4.2.1 Purpose and Architectural Role

Bridges raw network traffic and the ML processing layer: turns packets into a normalized, versioned flow-feature schema and publishes them reliably to the streaming platform.

4.2.2 Packet Capture and Handling

Uses Scapy to read packets from a live interface or replay a `.pcap` file; groups packets into flows keyed by the 5-tuple (source/destination IP, source/destination port, protocol); accumulates per-flow counters (packet/byte counts, SYN/FIN flags) until the flow is considered complete (FIN seen, or a timeout with no new packets).

4.2.3 Flow Feature Extraction and Normalization

Computes the 12-feature vector (duration, packet/byte counts each direction, mean packet length each direction, throughput, SYN/FIN counts, protocol) in the exact schema order the ML models were trained on, so live features and training features are never accidentally misaligned.

4.2.4 Data Validation and Publishing

Drops and logs malformed packets rather than crashing the pipeline; publishes each completed flow as a JSON event to the `network-flows` Redpanda topic within 2 seconds of flow completion; also persists the raw flow record to MongoDB regardless of eventual verdict, for retraining and audit purposes.

4.2.5 Fault Tolerance and Performance

If Redpanda is temporarily unreachable, the producer buffers events in memory up to a bounded limit and retries with backoff; if the buffer fills, oldest events are dropped and a warning is logged (traffic capture must never block on a downstream outage). Designed to sustain at least 500 flows/minute (SRS PERF-04).

4.3 AI Detection & Scoring Module

4.3.1 Purpose and Architectural Role

The detection core of the system: consumes flow events, produces an explainable benign/malicious verdict with severity, and hands the result to the data layer for the backend and frontend to act on.

4.3.2 Functional Responsibilities

Model loading at startup (supervised classifier, anomaly model, scaler, SHAP explainer); stream consumption; feature scaling; dual scoring; verdict fusion; explanation generation; schema-validated persistence.

4.3.3 Supervised and Anomaly Scoring

The supervised XGBoost classifier outputs a probability distribution over attack families (including Benign); the Isolation Forest anomaly model — trained only on benign flows — outputs an outlier flag and a continuous anomaly score, independent of the supervised model's training labels.

4.3.4 Feature Consistency with Training

The inference service uses the exact same `scaler.pkl` fit during training (Section 7.1) to transform live features, guaranteeing the live feature distribution matches what the models were trained on.

4.3.5 Verdict Fusion and Severity Assignment

See Section 7.3 for the fusion rule and Section 7.4 for severity/SHAP logic in detail.

4.3.6 Threat Intelligence Enrichment Hook

When a flow is scored malicious, the module (or the backend, on alert read) triggers a threat-intel lookup for the source IP, cached for 24 hours per IP.

4.3.7 Data Exchange with Backend

The ML service does not call the backend directly; it writes the completed alert document to MongoDB, and the backend's change-stream watcher picks it up (see 3.4.3). This one-way, database-mediated exchange keeps the two services independently deployable.

4.3.8 Performance and Reliability

Target: score a flow end-to-end (supervised + anomaly + SHAP) within 15 seconds (SRS PERF-08); on schema-invalid output or a crash mid-score, the event is retried up to 3 times before being logged as a processing failure and skipped, so one bad event cannot stall the whole consumer.

4.4 Alert Screening & Response Decision Support Module

4.4.1 Purpose and Architectural Role

Gives the analyst everything needed to triage an alert quickly and to act on it safely — this module owns the alert lifecycle state machine and the block-approval workflow.

4.4.2 Alert Review Workflow

Analyst opens the live feed → selects an alert → sees raw flow features, verdict, severity, SHAP explanation and (if available) threat-intel enrichment → decides to acknowledge, resolve, mark false positive, or approve a block.

4.4.3 Severity and Threshold Evaluation

Severity is derived from supervised confidence and anomaly strength against administrator-configurable thresholds (SRS FR-SCORE-04); alerts at or above the "critical" threshold also trigger a notification (Section 4.5).

4.4.4 Response Output and Analyst Decision

Every decision writes both a status update on the `Alert` document and, for block approvals, a new `BlockRule` row (Prisma/SQLite) recording the IP, reason, approving analyst and timestamp. The system itself never creates a `BlockRule` without an explicit analyst-initiated API call (SRS FR-RESP-05).

4.5 Notification Management Module

4.5.1 Purpose and Responsibilities

Delivers timely, non-duplicated awareness of new and changed alerts to analysts, both inside the app (Socket.io) and outside it (email/Slack for critical severity).

4.5.2 Notification Generation and Delivery

Triggered by the backend's MongoDB change-stream watcher on alert insert/update; in-app notifications are pushed immediately over Socket.io; critical-severity alerts additionally queue an email/Slack webhook call.

4.5.3 Notification Data Model

See Section 5.8 (`Notification` collection/table: userId, type, message, readFlag, createdAt).

4.5.4 Notification Lifecycle

Created (unread) → delivered (Socket.io push and/or webhook call) → read (analyst opens/dismisses it) → optionally auto-archived after 90 days (SRS data retention).

4.5.5 Duplicate Prevention

At most one notification per event per user: the change-stream handler is idempotent per alert `_id` + event type, so a retried change-stream event cannot create a second notification.

4.5.6 Performance Considerations

Notification dispatch runs asynchronously off the main request path so a slow email/Slack call never delays the Socket.io push or the API response.

4.6 Dashboard & Reporting Module

4.6.1 Purpose and Responsibilities

Gives analysts and administrators an at-a-glance operational picture and gives evaluators a way to inspect detection quality over time.

4.6.2 Dashboard Components

Live alert feed (virtualized list for performance), severity distribution chart, alert-volume-over-time chart, pipeline health indicators (capture/streaming/inference status).

4.6.3 Alert and Detection Visualization

Severity color-coding, a SHAP contribution bar chart per alert, and a rolling time-series of alerts raised, built with Recharts against data from `/api/stats/summary`.

4.6.4 Reporting and Analytics Functionality

Filterable by date range and severity; exposes model-evaluation metrics (accuracy, precision, recall, F1, confusion matrix) sourced from the offline training run (Section 7) for administrator/evaluator review.

4.6.5 Role-Based Access and Error Handling

The administrator dashboard additionally shows user counts and active block rules, gated by `authMiddleware`'s role check; all dashboard API calls degrade gracefully (empty-state UI) if a data source is temporarily unavailable rather than showing a raw error.

4.6.6 Performance Considerations

Summary statistics are computed with indexed MongoDB aggregation queries (Section 5.11) and cached for a few seconds server-side to keep repeated dashboard polling cheap.

4.7 Logging & Monitoring Module

4.7.1 Purpose and Responsibilities

Provides the immutable record of what the AI decided and what the analyst did about it, plus the operational visibility needed to know each service is healthy.

4.7.2 Logging Categories

Audit logs (ML verdicts, analyst decisions, admin actions — permanent, structured); **security logs** (login/logout/lockout — permanent); **operational logs** (errors, retries, health-check results — rotated).

4.7.3 Audit Log Storage and Monitoring

Stored in the `AuditLog` table (Prisma/SQLite) for transactional integrity and easy querying by actor/date; never editable or deletable through the application API (SRS FR-LOG-06).

4.7.4 Health Tracking and Failure Handling

Each service exposes a `/health` endpoint; Kubernetes readiness/liveness probes use these to restart unhealthy pods automatically (Section 8.4).

4.7.5 Log Retention and Security

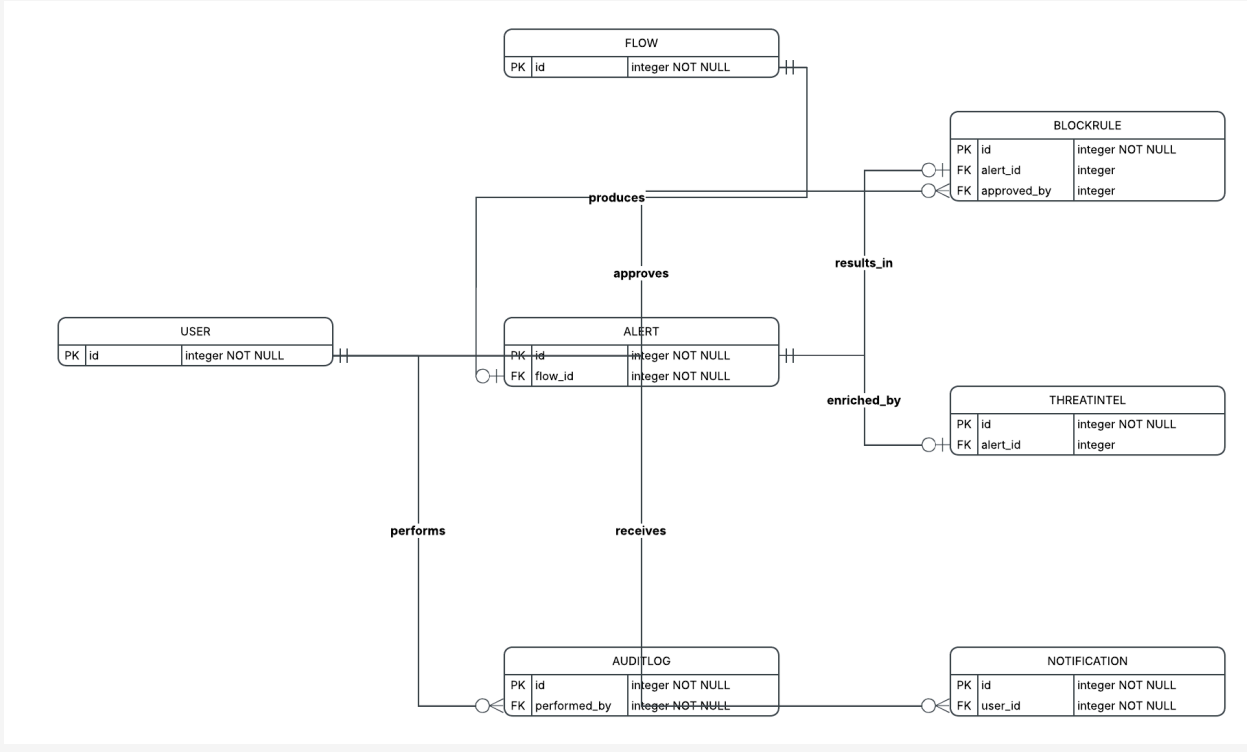
Operational logs are excluded from ever containing passwords, tokens or raw packet payload content (SRS SEC-07, FR-LOG-04); audit logs are retained at least 1 year per the SRS data-retention policy.

5. Data Design

5.1 Database Architecture Overview

Polyglot persistence: MongoDB for high-volume, semi-structured security data (flows, alerts, threat-intel cache); SQLite via Prisma for small, strictly-relational data (users, block rules, audit log) where transactional integrity matters more than throughput.

5.2 Database Schema Design



5.3 Users Table (SQLite / Prisma)

`id (PK)`, `email (unique)`, `passwordHash`, `role (ANALYST | ADMIN)`, `createdAt`.

5.4 Flows Collection (MongoDB)

`srcIp`, `dstIp`, `srcPort`, `dstPort`, `protocol`, `duration`, `packetCount`, `byteCount`, `flags`, `timestamp`. Indexed on `timestamp` and `srcIp` for time-range and per-IP queries.

5.5 Alerts Collection (MongoDB)

`flowId`, `srcIp`, `dstIp`, `supervisedLabel`, `supervisedConfidence`, `anomalyScore`, `verdict`, `severity`, `shapTopFeatures[]`, `status`, `assignedToUserId`, `createdAt`. Indexed on `severity`, `status` and `createdAt` for the live feed and filtered queries.

5.6 BlockRules Table (SQLite / Prisma)

`id (PK)`, `ip`, `reason`, `alertId` (Mongo ObjectId stored as string — no cross-database FK), `createdById (FK → User)`, `active`, `createdAt`.

5.7 ThreatIntel Collection (MongoDB)

`ip (unique)`, `abuseScore`, `country`, `lastChecked` — a cache to avoid redundant external API calls within 24 hours.

5.8 Notifications & AuditLog

Notification (MongoDB): `userId`, `type`, `message`, `readFlag`, `createdAt`. **AuditLog** (SQLite / Prisma): `id (PK)`, `actor`, `action`, `target`, `details`, `timestamp`.

5.9 Model Metadata (file-based, not a database table)

Model artifacts (`model_supervised.pkl`, `model_anomaly.pkl`, `scaler.pkl`, `shap_explainer.pkl`) are versioned as files with a version tag recorded on each `Alert` document (`modelVersion`), rather than as a database table, since they change only when the team retrains.

5.10 Data Relationships

- An alert always references exactly one flow (`flowId`).
- A block rule always references the alert that triggered it and the analyst who approved it.
- Deleting a flow record does not delete its associated audit records.
- Cross-database references (Mongo ObjectId stored inside a SQLite row, and vice versa) are resolved at the application layer, never assumed to be enforced by either database engine.

5.11 Indexing Strategy

- MongoDB: compound index on `{ severity: 1, status: 1, createdAt: -1 }` on `alerts` for the live feed's default sort/filter; single-field index on `flows.timestamp` for time-range queries.
- SQLite: unique index on `User.email`; index on `BlockRule.active` for the administrator's active-rules view.

5.12 Data Retention & Archival Strategy

- Raw flow records: 90 days, then archived or summarized.
- Alerts and audit logs: minimum 1 year.
- Notifications: purged after 90 days.
- Database backups: retained 30 days (Section 10.6).

6. Interface Design

6.1 User Interface Design

6.1.1 UI Design Principles

Severity is always communicated by color **and** text label (never color alone); every AI-generated score carries a visible "this is a recommendation" notice; loading and empty states are explicit, never a blank screen.

6.1.2 Analyst Dashboard Interface

Live alert feed (virtualized list, newest first), severity summary cards, alert-volume chart, quick filters (severity/status/source IP).

6.1.3 Administrator Dashboard Interface

User counts, active block-rule count, pipeline health indicators (capture/streaming/inference status), recent audit activity.

6.1.4 Alert Detail and Explanation Interface

Raw flow features, verdict and confidence, SHAP contribution bar chart, threat-intel enrichment panel, action buttons (Acknowledge, Resolve, False Positive, Approve Block).

6.1.5 Live Alert Feed Interface

Real-time insertion of new alerts via Socket.io without a page refresh; smooth status-badge updates when another analyst acts on the same alert.

6.1.6 Block-Rule Management Interface

Administrator view of all active/inactive block rules with IP, reason, approving analyst and timestamp; toggle to deactivate a rule.

6.1.7 Reporting Interface

Date-range and severity filters over alert-volume and model-performance charts; CSV export of the current filtered view.

6.1.8 Role-Based UI Rendering

Navigation and route access are both gated by role: the router checks the decoded JWT role before rendering an Analyst- or Administrator-only route, in addition to the backend's own authorization checks (defense in depth).

6.2 API Design

6.2.1 API Architecture

REST over HTTPS/JSON for CRUD; WebSocket (Socket.io) for real-time alert push; grouped by resource (Auth, Alerts, BlockRules, ThreatIntel, Stats, Admin), each with its own Express router.

6.2.2 Authentication APIs

`POST /api/auth/register`, `POST /api/auth/login`, `POST /api/auth/refresh`, `POST /api/auth/logout`.

6.2.3 Alert APIs

`GET /api/alerts` (filterable), `GET /api/alerts/:id`, `PATCH /api/alerts/:id/status`, `POST /api/alerts/:id/block`.

6.2.4 Block-Rule APIs

`GET /api/blockrules`, `PATCH /api/blockrules/:id` (activate/deactivate).

6.2.5 Threat-Intelligence APIs

`GET /api/threatintel/:ip` (cached or fresh lookup).

6.2.6 Statistics and Reporting APIs

`GET /api/stats/summary` (counts by severity/status), `GET /api/stats/model-performance` (accuracy/precision/recall/F1 from the last training run).

6.2.7 Administration APIs

`GET/PATCH /api/admin/users` (suspend/reactivate), `PATCH /api/admin/config` (severity thresholds).

6.2.8 Audit APIs

`GET /api/admin/audit-logs` (filterable, read-only).

6.3 External Interfaces

6.3.1 Streaming Platform Interface

Kafka-protocol producer (sniffer) and consumer (ml-service) against the `network-flows` topic on Redpanda.

6.3.2 Threat Intelligence Provider Interface

REST call to a provider such as AbuseIPDB with the source IP; response cached in the `ThreatIntel` collection for 24 hours.

6.3.3 Email/Slack Notification Interface

Outbound webhook call for critical-severity alerts; delivery failures are logged, not retried indefinitely, so a notification outage never blocks the core detection pipeline.

6.3.4 Container and Deployment Infrastructure Interface

Docker Engine (local), Kubernetes API (cluster deployment), Jenkins (CI/CD orchestration) — see Section 10 for full detail.

7. Detailed Algorithm Design

7.1 Flow Feature Extraction Algorithm

7.1.1 Objective

Convert raw packets into a fixed-length numeric feature vector that matches the schema the ML models were trained on.

7.1.2 Input Data and Processing Model

Input: a live packet stream or `.pcap` file. Packets are grouped by the 5-tuple key (srcIp, dstIp, srcPort, dstPort, protocol) into an in-memory flow table.

7.1.3 Flow Assembly Pipeline

For each packet: look up or create the flow entry → update running counters (packet count, byte count per direction, SYN/FIN flags, first/last timestamp) → on flow completion (FIN flag or idle timeout), emit the flow record and remove it from the table.

7.1.4 Feature Computation

Derived features: `flow_duration` (last − first timestamp, µs), `flow_bytes_per_sec` and `flow_packets_per_sec` (throughput), `fwd/bwd_packet_length_mean` (bytes ÷ packets per direction). See the project roadmap document's ML Pipeline tab for the exact 12-feature list and reference implementation.

7.1.5 Structured Output Generation

Output: a JSON object `{ srcIp, dstIp, srcPort, dstPort, protocol, features: [...] }` published to the `network-flows` topic — the same shape the training pipeline's `preprocess.py` produces from the labeled CSV data, so live and offline features are structurally identical.

7.2 Supervised Attack Classification Algorithm

7.2.1 Objective

Name the attack family for a scored flow (or classify it Benign), with a confidence value.

7.2.2 Model and Training Data

XGBoost multi-class classifier (`objective=multi:softprob`), trained on CIC-IDS2017, validated for generalization on UNSW-NB15 (held out, never trained on).

7.2.3 Inference Pipeline

Scale the incoming feature vector with the training-fit `StandardScaler` → `predict_proba` → take the argmax class and its probability as `(supervisedLabel, supervisedConfidence)`.

7.2.4 Output

`{ supervisedLabel: "DDoS", supervisedConfidence: 0.94 }` — attached to the eventual alert document.

7.3 Anomaly Detection and Verdict Fusion Algorithm

7.3.1 Objective

Catch statistically unusual traffic the supervised model was never trained to recognize (zero-day-like behavior), and combine both signals into one decision.

7.3.2 Anomaly Model

Isolation Forest, trained **only** on benign-labeled rows from the training set, so it has no exposure to known attack patterns and models "normal" traffic shape directly.

7.3.3 Verdict Fusion Rule

A flow is `malicious` if **either**: (a) the supervised model's top class is not Benign, **or** (b) the Isolation Forest flags the flow as an outlier (`predict(X) == -1`). This "OR" fusion — rather than requiring both models to agree — is a deliberate design choice: it maximizes recall on the anomaly path, favoring "flag for analyst review" over "silently miss a novel attack", which fits the SRS requirement that the AI only recommends and the analyst always makes the final call.

7.3.4 Severity Assignment

`severity = critical` if supervised confidence > 0.9 and the class is a known attack; `high` if malicious by either signal; `low` otherwise — thresholds are administrator-configurable (SRS FR-SCORE-04).

7.4 Explainable Scoring Algorithm

7.4.1 Objective

Make every verdict auditable and understandable to a non-ML-specialist analyst.

7.4.2 SHAP Computation

A `TreeExplainer` (fast, exact for tree ensembles like XGBoost) computes per-feature SHAP values for the predicted class; the 3 features with the largest absolute contribution are kept.

7.4.3 Explanation Rendering

Each retained feature is paired with its contribution magnitude and sign (positive = pushed toward malicious, negative = pushed toward benign), rendered as a colored bar in the frontend (Section 6.1.4).

7.4.4 Auditability

The SHAP output, supervised/anomaly scores and model version are all persisted on the alert document, so any past verdict can be fully reconstructed and explained later, even after the model is retrained.

7.5 Alert Triage and Ranking Algorithm

7.5.1 Objective

Surface the alerts that most need analyst attention first, without ever making the analyst's decision for them.

7.5.2 Alert Filtering

The live feed query supports filtering by severity, status and source IP, implemented as indexed MongoDB queries (Section 5.11).

7.5.3 Alert Ranking

Default sort: severity (critical → low) then recency (newest first); ties broken by supervised confidence, highest first.

7.5.4 Analyst Decision Support

Each alert surfaces its explanation, threat-intel enrichment and related alerts (same source IP) directly in the detail view, so the analyst has full context in one screen rather than needing to cross-reference other tools.

7.5.5 Human Decision Control

No algorithm in this system is permitted to create a `BlockRule`, change an alert's terminal status, or take any external action; every state-changing action listed in this section requires an authenticated analyst-initiated API call (SRS FR-RESP-05).

8. Non-Functional Design

8.1 Overview

This section maps each non-functional SRS requirement group to the design decision that satisfies it.

8.2 Performance Design

8.2.1 Response Time Requirements

Express routes are kept thin (validation → DB call → response); heavy aggregation queries (`/api/stats/summary`) use indexed MongoDB pipelines rather than in-application loops, targeting the SRS's 2-second/95th-percentile budget (PERF-01).

8.2.2 Flow Ingestion Performance

Feature extraction runs as a lightweight, allocation-conscious loop per packet; the sniffer publishes asynchronously so a slow Redpanda write never blocks packet reading (PERF-04).

8.2.3 ML Inference Performance

Models are loaded once at service startup (not per-request); SHAP's `TreeExplainer` is used specifically because it is fast for tree ensembles, keeping the 15-second per-flow budget achievable (PERF-08).

8.2.4 Performance Optimization Strategies

Compound MongoDB indexes for the live feed's default query shape (Section 5.11); debounced `stats-updated` Socket.io events (at most 1/sec) so a burst of alerts doesn't flood connected dashboards.

8.3 Scalability Design

8.3.1 Frontend and Backend Scaling

Both are stateless (Section 3.6.4) and run as multiple Kubernetes replicas behind a Service, scaled horizontally by adding pods.

8.3.2 ML Service Scaling

A Horizontal Pod Autoscaler targets 70% CPU utilization on `ml-service` (the most compute-intensive component), scaling 1–3 replicas.

8.3.3 Database Scalability

MongoDB collections are designed with indexes that support future sharding by `srcIp` or time range if flow volume grows beyond a single-node capacity; SQLite is intentionally kept for low-volume relational data only, avoiding its single-writer limitation from becoming a bottleneck.

8.3.4 Streaming Layer Scalability

Redpanda topics can be partitioned by source IP hash to allow multiple `ml-service` consumer instances to process flows in parallel in a future iteration.

8.4 Availability & Reliability

8.4.1 Availability Strategy

Kubernetes readiness/liveness probes against each service's `/health` endpoint keep only healthy pods receiving traffic (AVL-01, AVL-02).

8.4.2 Fault Tolerance

Each layer degrades independently: an ML outage queues events in Redpanda rather than losing them; a notification outage is logged, not retried indefinitely, so it can't back up the pipeline (REL-02).

8.4.3 ML Service Failure Handling

Inference failures (timeout, schema mismatch) are retried up to 3 times with backoff, then logged as a processing failure and skipped — never silently dropped without a trace (REL-03).

8.4.4 Background Job Retry and Recovery

The Redpanda consumer's offset is only committed after a successful, schema-validated write to MongoDB, so a crash mid-processing re-delivers the event rather than losing it.

8.5 Maintainability Design

Separate repositories-in-one-monorepo structure (Section 4 of the roadmap document) with clear service boundaries; linting (ESLint/Flake8) enforced in CI; ≥70% test coverage target on core modules (MNT-03).

8.6 Usability Design

Severity always shown with both color and label; every score carries an explicit "recommendation, not a decision" notice (USE-04); the highest-severity alert is reachable within 3 clicks of the dashboard home (USE-02).

8.7 Portability Design

All configuration (URLs, secrets, capture interface) is environment-variable driven so the identical container images run unmodified in Docker Compose and Kubernetes (POR-01).

9. Security Design

9.1 Security Architecture Overview

Defense in depth: transport encryption, authenticated and authorized API access, validated inputs and outputs (including ML outputs), least-privilege service boundaries, and an immutable audit trail.

9.2 Authentication Design

9.2.1 JWT-Based Authentication

15-minute access tokens, 7-day refresh tokens; access tokens are never persisted client-side beyond memory, limiting exposure from XSS.

9.2.2 Password Security

bcrypt hashing (cost factor 12); minimum complexity enforced server-side (SRS FR-AUTH-02).

9.2.3 Refresh Token Management

Stored in an httpOnly, secure cookie; invalidated server-side on logout.

9.2.4 Account Lockout

5 failed attempts within 15 minutes locks the account for 15 minutes (FR-AUTH-07).

9.2.5 WebSocket Authentication

The same JWT is validated at the Socket.io handshake (`io.use(verifySocketToken)`) before any event subscription is allowed.

9.3 Authorization and RBAC

Every protected route declares its allowed role(s); `authMiddleware` rejects any request whose JWT role doesn't match, regardless of what the client believes it's allowed to do.

9.4 Communication Security

9.4.1 HTTPS and TLS

All external traffic uses TLS 1.2+; the Kubernetes ingress terminates TLS in front of the frontend and backend.

9.4.2 Secure Internal Service Communication

MongoDB, Redpanda and the ML service are not exposed outside the cluster's internal network (Section 3.3.5).

9.4.3 CORS and Security Headers

CORS restricted to the known frontend origin; standard security headers (helmet.js defaults: `X-Content-Type-Options`, `X-Frame-Options`, etc.) applied to every response.

9.5 Data Security

9.5.1 Data at Rest

Database volumes are container-managed volumes with restricted host access; production secrets are never baked into images.

9.5.2 Data in Transit

All browser ↔ backend and (in production) inter-service traffic is TLS-encrypted.

9.5.3 Flow Data Minimization

Only header-derived flow features are stored — no packet payload content is ever captured or persisted (SRS SEC-07), minimizing sensitive-data exposure by design.

9.5.4 Data Retention and Deletion

Per Section 5.12; expired records are purged by a scheduled job rather than kept indefinitely.

9.6 API Security Controls

9.6.1 Input Validation and Sanitization

All request bodies validated against a schema (e.g. Zod/Joi) before touching business logic.

9.6.2 Rate Limiting

Per-IP rate limiting on all `/api/auth/*` routes to blunt credential-stuffing attempts.

9.6.3 NoSQL Injection and XSS Protection

Mongoose schema typing plus parameterized queries prevent operator injection; React's default escaping plus a strict Content-Security-Policy mitigate XSS.

9.6.4 ML Output Schema Validation

Every inference-service output is validated against a fixed JSON schema before being persisted (SRS FR-DETECT-07) — an invalid or malformed model output can never silently corrupt an alert record.

9.6.5 Model and Dependency Scanning

Container images and Python/Node dependencies are scanned for known vulnerabilities in the Jenkins pipeline (SEC-06) before deployment.

9.7 Logging and Audit Security

9.7.1 Security Event Logging

Login, logout, failed login and lockout events are recorded with timestamp and (hashed/partial) IP.

9.7.2 Audit Trail Protection

The `AuditLog` table has no application-level update or delete endpoint — it is append-only by design (FR-LOG-06).

9.7.3 Sensitive Data Exclusion

Logs never contain passwords, tokens, or raw packet content (FR-LOG-04).

9.7.4 Audit Log Retention

Minimum 1 year, per Section 5.12.

10. Deployment & Operations

10.1 Deployment Architecture

Six containerized services (`sniffer`, `ml-service`, `backend`, `frontend`, `mongo`, `redpanda`) composed locally with Docker Compose and deployed to Kubernetes (Minikube/Kind for the capstone demo) for the final release.

10.2 Deployable Units

10.2.1 Frontend Container

Multi-stage build: Vite production build served by nginx.

10.2.2 Backend API Container

Node.js 24-alpine base; runs `prisma generate` at build time.

10.2.3 ML Service Container

Python 3.12-slim base; bundles trained model artifacts (`.pkl` files) copied in at build time.

10.2.4 Database Deployment

MongoDB as a StatefulSet-equivalent single-replica deployment with a persistent volume (replica set enabled for change-stream support); SQLite as a file on the backend container's persistent volume.

10.3 Docker Configuration

10.3.1 Container Configuration

Each service has its own `Dockerfile` (see the project roadmap document's DevOps tab for the exact files).

10.3.2 Environment Configuration

All secrets and connection strings (`MONGO_URL`, `JWT_SECRET`, streaming broker address) are injected via environment variables / Kubernetes ConfigMaps and Secrets — never hardcoded.

10.3.3 Docker Compose Configuration

A single `docker-compose.yml` wires all six services together for local development, with `mongo-init` running `rs.initiate()` once so change streams work.

10.3.4 Container Health Checks

Each service's `/health` endpoint backs a Docker healthcheck and, in Kubernetes, a readiness/liveness probe.

10.4 CI/CD Pipeline



10.4.1 Source Code Management

Git, with feature branches merged to `main` via pull request; Jira issue keys referenced in commit messages for traceability.

10.4.2 Build and Test Pipeline

Jenkins runs backend unit tests and the frontend build on every push.

10.4.3 Security and Dependency Scanning

Container images and dependency manifests are scanned before push (SEC-06).

10.4.4 Container Build and Deployment

Each of the 4 application images (`backend`, `frontend`, `ml-service`, `sniffer`) is built and pushed with a build-number tag.

10.4.5 Kubernetes Deployment

`kubectl set image` rolls out the new image tag per deployment, followed by `kubectl rollout status` to confirm success before the pipeline reports green.

10.5 Monitoring & Operations

10.5.1 Application Monitoring

Structured request logs from the backend; browser-side error boundaries in the frontend.

10.5.2 Service Health Monitoring

Kubernetes probes plus a lightweight external check hitting each `/health` endpoint on an interval.

10.5.3 ML Processing Monitoring

Processing-failure counts and average scoring latency are tracked and surfaced on the administrator dashboard (Section 4.6).

10.5.4 Error and Failure Monitoring

Unhandled exceptions in any service are logged with a stack trace (server-side only) and a generic message returned to the client (SRS UI-05).

10.5.5 Operational Logging

Centralized via each container's stdout, collected by the container runtime/Kubernetes logging layer.

10.6 Backup & Disaster Recovery

10.6.1 Database Backup Strategy

Scheduled `mongodump`/SQLite file snapshots to a persistent volume outside the application containers.

10.6.2 Backup Retention

30 days, per the SRS data-retention policy.

10.6.3 Service Recovery

Kubernetes automatically restarts crashed pods (AVL-02); stateful data (Mongo, SQLite) survives pod restarts via persistent volumes.

10.6.4 Disaster Recovery Considerations

For the capstone scope, disaster recovery is limited to restoring from the most recent backup; multi-region failover is out of scope (see SRS 2.6, future versions).

11. Testing

11.1 Testing Overview

Every requirement in the SRS traceability matrix (Section 6.5 there) maps to at least one test case here.

11.2 Unit Testing

Backend route handlers, Mongoose/Prisma model validation, ML feature-extraction and fusion-rule logic — each tested in isolation with mocked dependencies.

11.3 Integration Testing

Sniffer → Redpanda → ml-service → MongoDB, exercised end to end with a small sample `.pcap` in a test-only Compose stack.

11.4 API & Service Testing

Every REST endpoint tested for both the happy path and the documented error codes (400/401/403/404/409/429/500).

11.5 ML Processing and Detection Testing

Held-out CIC-IDS2017 test split for supervised metrics; UNSW-NB15 for generalization; a labeled set of known-benign flows to measure the anomaly model's false-positive rate.

11.6 Performance Testing

Load test with 50 concurrent simulated dashboard connections (PERF-06); flow-scoring latency measured under sustained 500 flows/minute ingestion (PERF-04, PERF-08).

11.7 Security Testing

NoSQL injection and XSS payloads against all input fields; rate-limit trigger verification; container image vulnerability scan review.

11.8 Reliability and Failure Recovery Testing

Kill the ML service mid-run and confirm the rest of the platform keeps working and queued events are processed on recovery (REL-02, REL-03); kill a backend pod and confirm Kubernetes restarts it within 2 minutes (AVL-02).

11.9 User Acceptance Testing (UAT)

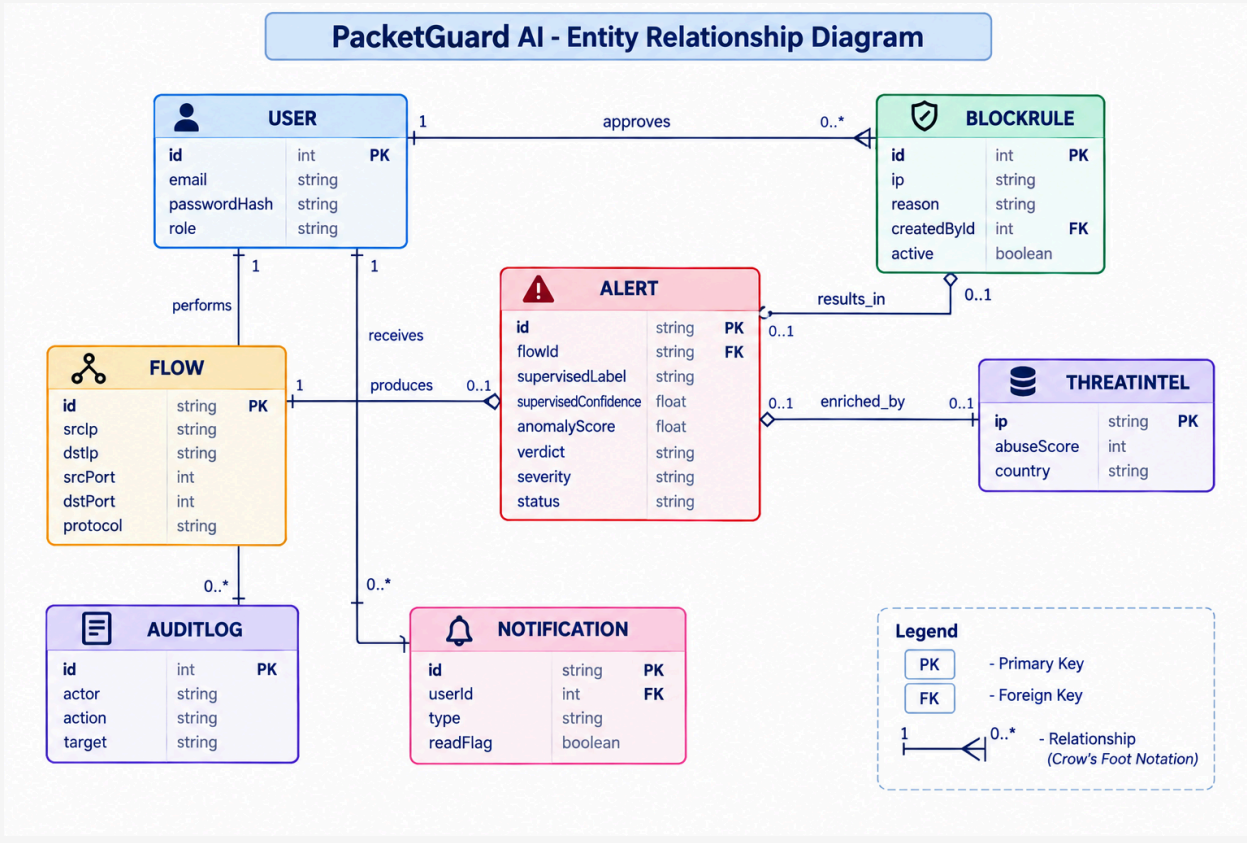
A scripted analyst walkthrough: log in → see a live alert appear from a replayed attack pcap → read the SHAP explanation → approve a block → confirm the status updates live in a second browser tab.

12. References

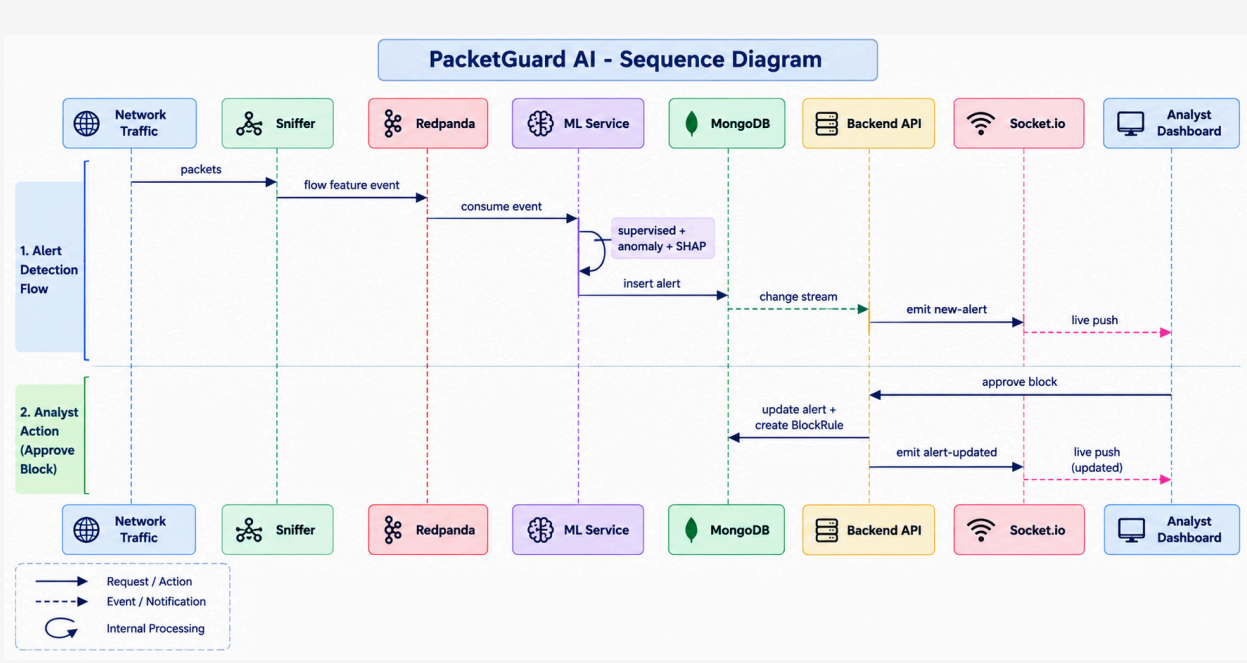
1. IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications
2. ISO/IEC/IEEE 29148:2018, Requirements Engineering
3. PacketGuard AI Software Requirements Specification (companion document)
4. PacketGuard AI capstone roadmap document (internal project reference)
5. CIC-IDS2017 and UNSW-NB15 dataset documentation
6. OWASP Top 10
7. Official documentation of React, Node.js, Express.js, MongoDB, FastAPI, Docker and Kubernetes
8. scikit-learn, XGBoost and SHAP library documentation

13. Appendices

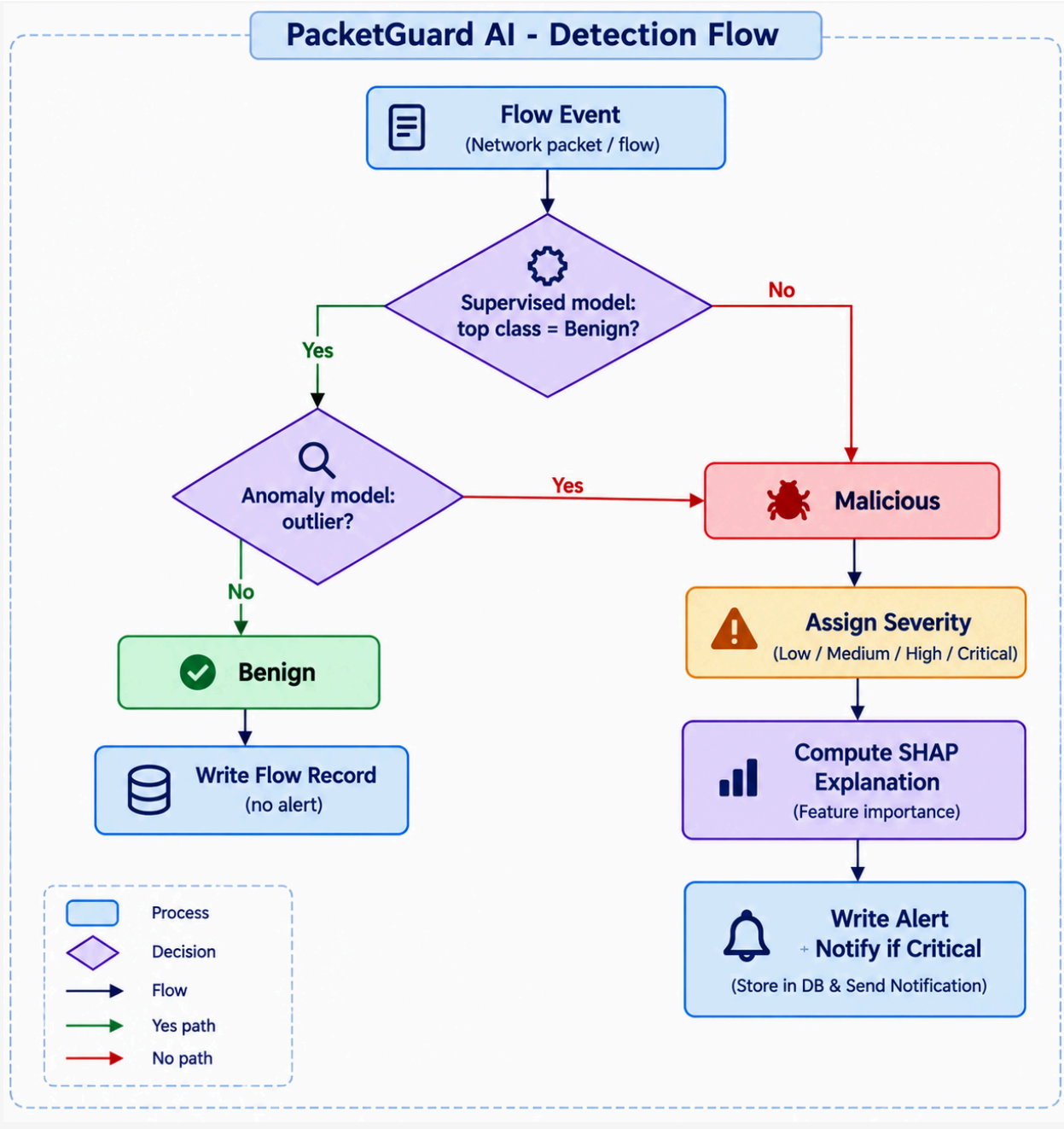
13.1 Appendix A: ER Diagram (Logical View)



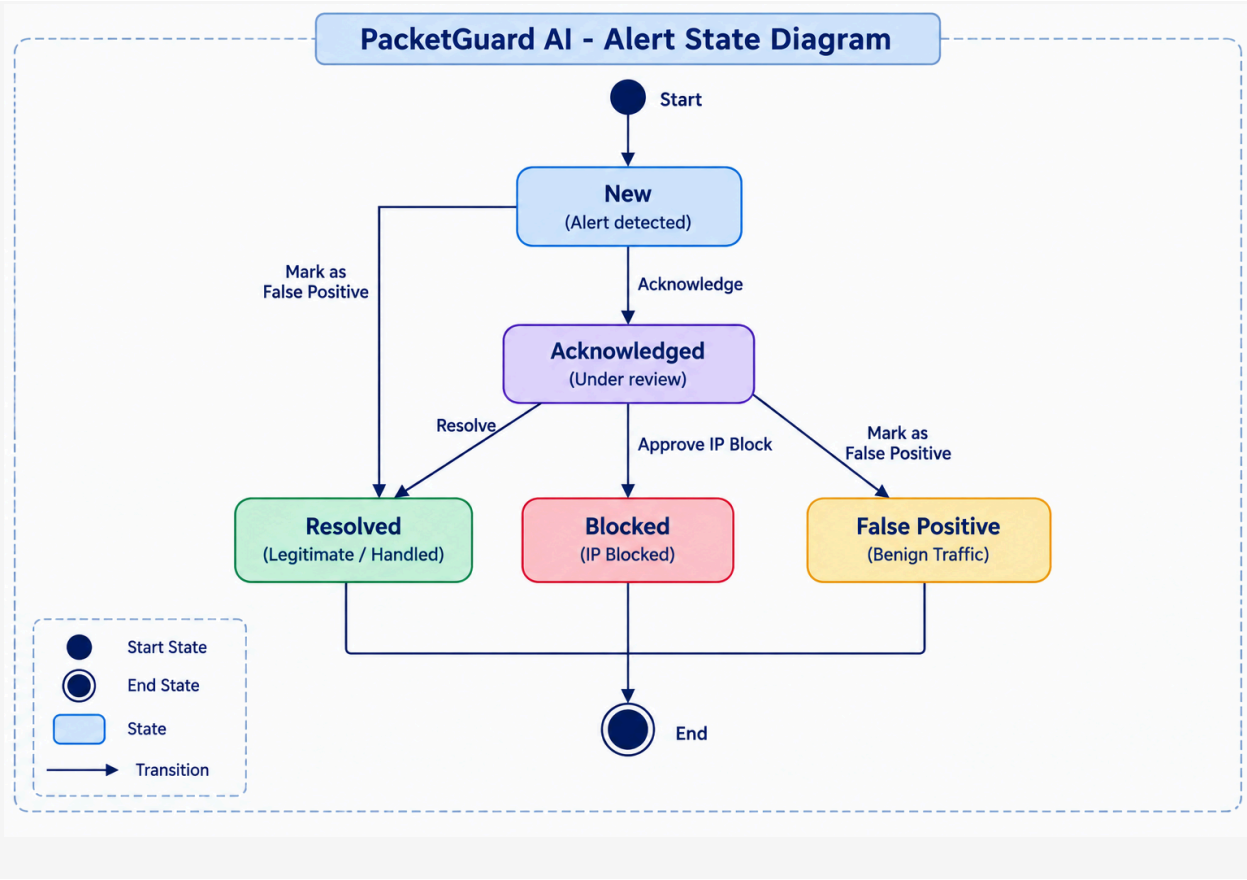
13.2 Appendix B: High-Level System Sequence Diagram



13.3 Appendix C: Detection & Alert Flow Diagram



13.4 Appendix D: Alert Status Transition Diagram



13.5 Appendix E: API Request and Response Format Examples

Login (`POST /api/auth/login`)

```
{ "email": "analyst@example.com", "password": "Str0ngPass1" }
```

Response:

```
{ "accessToken": "<jwt>", "refreshToken": "<token>", "user": { "id": 1, "role":
"analyst" } }
```

List alerts (`GET /api/alerts?severity=critical`)

```
{
  "alerts": [
    {
      "id": "a_9001",
      "srcIp": "203.0.113.55",
      "supervisedLabel": "DDoS",
      "severity": "critical",
      "status": "new"
    }
  ]
}
```

Approve a block (`POST /api/alerts/:id/block`)

```
{ "reason": "Confirmed DDoS source" }
```

Response:

```
{ "blockRuleId": 501, "ip": "203.0.113.55", "active": true }
```

Error format

```
{ "error": { "code": "INVALID_STATUS_TRANSITION", "message": "Cannot move an alert from 'resolved' back to 'new'." } }
```

For the full API surface, see this document's Section 6.2 and the SRS's Appendix F.